Glyph2Hand

分层参考编码与条件流匹配：架构图与实现说明

2026 年 10 月 10 日　当前代码、作者风格损失与训练约定

1　任务、范围与核心假设

根据同一书写者的若干参考字，生成指定汉字的手写图。输入为

$$\mathcal{C} = \{(H_i, K_i)\}_{i=1}^{N}, \qquad K_\star,$$

其中 H_i 是参考手写图，K_i 是同一个汉字的标准字体图；$K_\star$ 是目标汉字的标准字体图。实际标准字字体由数据准备参数 `--font` 指定；训练说明使用文津宋体 `WenJinMinchoP0-Regular.ttf`，代码不硬编码微软雅黑。图中的标准侧使用微软雅黑、手写侧使用楷书字形，仅作示意，不代表训练样本；符号 K 仅作为标准字图的记号。一组指一个对应的 (H_i, K_i)，不是一张独立图像。所有参考手写图来自同一书写者，$N \geq 1$。输出 $\widehat{H}_\star$ 应同时保持目标字内容和参考笔迹风格。训练真值 $H_\star$ 不进入参考集合，也不作为生成网络的独立条件输入。

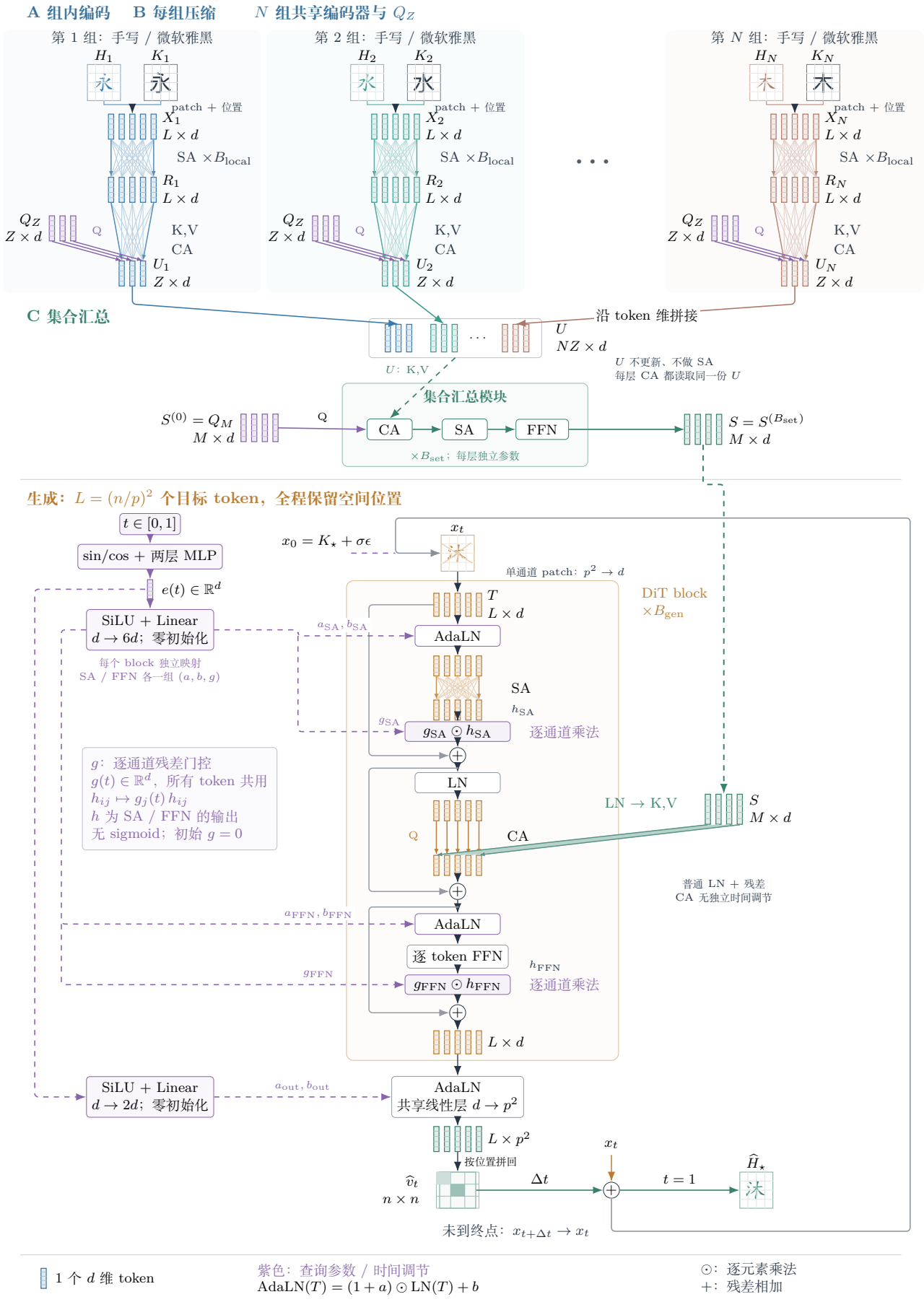
本版参考分支采用以下顺序：

1. 每组独立编码，在整字上下文中理解局部形状以及手写与标准字之间的关系。
2. 每组压缩为 Z 个语义描述 token，表示从该参考字中观察到的写法证据。
3. 由 M 个可学习查询初始化风格 token，经过 B_{set} 个集合汇总块；每层依次用交叉注意力读取全部 NZ 个描述、在 M 个风格 token 间做自注意力，再经过前馈网络。

核心假设：组内结构已被编码进 Z 个描述，后续融合可以主要依据描述内容，不再显式依赖原始 patch 的分组关系。这不意味着字符内容被完全消除，也不保证每个 token 对应一种笔画。笔画、部件及其位置仍可能是解释某种写法所必需的信息。这一假设和压缩容量需要通过实验验证；本文记录 `src` 中的实现行为，不提供生成质量或收敛效果结论。完整字段、命令行、日志和异常处理约定见上一级 `README.md`，服务器操作见 `TRAINING.md`。`README.v1-with-diagrams.*` 是历史设计，不作为当前实现依据。

生成分支暂按“目标标准字加噪声作为起点”的像素空间 flow 写出完整公式。目标标准字仅用于构造起点，不作为主干的独立提示通道；速度网络的输入为当前状态 x_t、时间 t 和参考风格 S。噪声强度、各层数和 token 数量均为可调实验超参数，尚未由实验确定。

架构总览　参考集合压缩与条件 flow



条带数量为示意；实际数量以 L, Z, M, NZ 为准。连线表示注意力读取关系；QKV 在 MHA 内独立投影。图中字形仅作示意；实际标准字由数据准备所选字体渲染。生成主干的 SA、FFN 采用 AdaLN-Zero，生成 CA 采用普通 LN 与残差；集合汇总的 CA、SA、FFN 均采用普通前置 LN 与残差。参考逐组编码的归一化、残差及 FFN 从略；最终字图经过灰度映射。

2 统一符号与数据形状

所有图像先映射至 $[-1,1]$，采用 $n\times n$ 单通道灰度图。块边长为 p，要求 $p\mid n$。每边块数为 $q=n/p$，每组块数为 $L=q^2$。这里使用 q 表示每边块数，避免与 flow 状态 x_t 混淆。下表省略批次维；每行特征都是一个 d 维 token。

符号	形状 / 含义	职责
N	参考对数量	随输入变化，至少为 1
Z	每组描述数量	单字证据容量，固定超参数
M	最终风格数量	整个参考集合的输出容量，固定超参数
B_{set}	集合汇总层数	每层依次执行 CA、SA、FFN；代码默认 2，至少为 1
X_i	$L\times d$	第 i 组的初始 patch 特征
R_i	$L\times d$	经过组内注意力的空间特征
Q_Z	$Z\times d$	所有组共用的可学习压缩查询
U_i	$Z\times d$	第 i 组的语义描述
U	$NZ\times d$	全部组描述沿行拼接，直接提供键和值
Q_M	$M\times d$	可学习的集合汇总查询
S	$M\times d$	最终风格条件
$T^{(b)}$	$L\times d$	生成主干第 b 层的目标特征
$x_t,\widehat{v}_t$	$n\times n\times 1$	当前生成状态、预测像素速度

Z 和 M 都表示 token 数量，不表示特征维度；它们不必相等，也没有预设大小关系。Q_Z,Q_M 是训练得到的模型参数，不是每次生成时重新采样的随机量。

3 参考分支：从成对字图到固定风格表示

3.1 注意力块的具体约定

为使下文公式没有歧义，本文采用带残差、前置层归一化和前馈网络的注意力块。交叉注意力块 $\mathrm{CA}(A,B)$ 由 A 提供查询，B 提供键和值：

$$\begin{aligned}\widetilde{A}&=A+\mathrm{MHA}(\mathrm{LN}_A(A),\mathrm{LN}_B(B),\mathrm{LN}_B(B)),\\ \mathrm{CA}(A,B)&=\widetilde{A}+\mathrm{FFN}(\mathrm{LN}_F(\widetilde{A})).\end{aligned}$$

自注意力块 $\mathrm{SA}(A)$ 为

$$\begin{aligned}\widetilde{A}&=A+\mathrm{MHA}(\mathrm{LN}_A(A),\mathrm{LN}_A(A),\mathrm{LN}_A(A)),\\ \mathrm{SA}(A)&=\widetilde{A}+\mathrm{FFN}(\mathrm{LN}_F(\widetilde{A})).\end{aligned}$$

这里的 LN 是逐 token 的层归一化，不是 QKV 投影。$\mathrm{MHA}(X_Q,X_K,X_V)$ 的三个参数表示投影前的特征来源，模块内部再进行各头的 QKV 线性投影。以第 h 个头为例，省略可选偏置：

$$\begin{aligned}Q_h&=X_QW_Q^{(h)},\qquad K_h=X_KW_K^{(h)},\qquad V_h=X_VW_V^{(h)},\\ O_h&=\mathrm{softmax}\!\left(\frac{Q_hK_h^{\top}}{\sqrt{d_h}}\right)V_h,\\ \mathrm{MHA}(X_Q,X_K,X_V)&=\mathrm{Concat}_h(O_h)W_O.\end{aligned}$$

d_h 是每头维度，W_O 是多头结果拼接后的输出投影。本稿的 $W_Q^{(h)},W_K^{(h)},W_V^{(h)}$ 分别参数化，不共享权重。因此自注意力中三个输入均为 $\mathrm{LN}_A(A)$，仅表示 Q、K、V 来自同一份特征，不施加投影后的 $Q_h=K_h=V_h$ 约束。同理，交叉注意力中键和值均来自 $\mathrm{LN}_B(B)$，也不表示 $K_h=V_h$。

与共享键值的区别：在本稿所用的相同键值维度下，若将 K、V 投影及其偏置（如有）绑定，或直接令 $V_h=K_h$，才是显式约束投影后 $K=V$ 的另一种结构；本稿不采用该约束。上述 QKV 投影约定也适用于生成主干中的多头注意力。前馈网络暂用两层线性映射和 GELU；归一化仅沿特征维，不跨 token 或参考组。不同阶段、不同层拥有各自参数；只有明确说明的逐组操作共享参数。

下文“注意力池化”采用 Set Transformer 的 PMA 思想：固定数量的可学习查询读取可变数量的输入。这里给出了本稿使用的具体块结构，不宣称逐项复现原论文的归一化和前馈配置。

3.2 阶段 A：组内分块与上下文编码

将一组 H_i, K_i 沿通道拼接为 $n \times n \times 2$。同位置的两个 $p \times p$ 小块一起展开，得到 $2p^2$ 维向量，再投影到 d 维。这不是把手写和标准字混合平均，也不是分别产生两套无配对关系的 token。定义

$$X_i = \mathrm{PatchEmbed}_{\mathrm{ref}}([H_i, K_i]) + P_{xy} \in \mathbb{R}^{L \times d}, \qquad R_i = E_{\mathrm{local}}(X_i).$$

$P_{xy} \in \mathbb{R}^{L \times d}$ 为图内二维位置编码；暂用固定二维正弦余弦编码。E_{local} 由 B_{local} 层 SA 组成。每组的 L 个 token 可以互相读取，但此阶段不读取其他参考组。所有组使用相同的分块投影、位置编码和编码器参数。

这一阶段保留 L 个空间位置。目的在于先获得带整字上下文的局部描述，为跨字比较提供依据；仅添加二维位置、然后直接把原始 patch 混合，不能完整表达哪些 patch 属于同一个字。组内编码也不保证自动学到可解释的笔画分类。

3.3 阶段 B：每组压缩为 Z 个语义描述

用同一套可学习查询 $Q_Z \in \mathbb{R}^{Z \times d}$ 独立读取各组：

$$U_i = \mathrm{CA}_{\mathrm{local\ pool}}(Q_Z, R_i) \in \mathbb{R}^{Z \times d}.$$

查询来自 Q_Z，键和值来自第 i 组的 R_i。每个查询均可读取该组的全部 L 个空间特征；输出位置对应查询槽位，不再对应固定图像块。各组之间共享 Q_Z 和此注意力块的全部参数。

本稿采用一次上述压缩，随后拼接各组描述，供 M 个风格 token 在多层集合汇总中反复读取。压缩后的 U_i 不再进行组内或跨组自注意力。压缩后不再保留 R_i 到主干或最终池化的旁路，因而 Z 是真实的信息瓶颈。

相同查询槽位为跨字描述提供共同的学习起点，但不强制语义完全对齐。例如不能预先断言“第一个 token 永远表示横画”。这里的 U_i 是来自单字的证据：有些写法在该字中没有出现，模型需要在后续集合融合中结合其他字。压缩不会自动实现内容与风格的严格解耦，也不要求每个描述能独立解释。

3.4 阶段 C：多层 CA–SA–FFN 汇总为 M 个风格 token

将各组描述沿 token 维拼接，作为每个集合汇总层的固定键值来源：

$$U = [U_1; \ldots; U_N] \in \mathbb{R}^{NZ \times d}, \qquad S^{(0)} = Q_M \in \mathbb{R}^{M \times d}.$$

设集合汇总深度为 $B_{\mathrm{set}} \geq 1$，当前代码默认 $B_{\mathrm{set}} = 2$。第 b 层先由当前风格 token 读取 U，再在 M 个风格 token 之间进行自注意力，最后执行逐 token 前馈网络：

$$\begin{aligned}
C^{(b)} &= S^{(b)} + \mathrm{MHA}^{(b)}_{\mathrm{ca}}\left(\mathrm{LN}^{(b)}_{q}(S^{(b)}), \mathrm{LN}^{(b)}_{u}(U), \mathrm{LN}^{(b)}_{u}(U)\right), \\
A^{(b)} &= C^{(b)} + \mathrm{MHA}^{(b)}_{\mathrm{sa}}\left(\mathrm{LN}^{(b)}_{s}(C^{(b)}), \mathrm{LN}^{(b)}_{s}(C^{(b)}), \mathrm{LN}^{(b)}_{s}(C^{(b)})\right), \\
S^{(b+1)} &= A^{(b)} + \mathrm{FFN}^{(b)}\left(\mathrm{LN}^{(b)}_{f}(A^{(b)})\right), \qquad b = 0, \ldots, B_{\mathrm{set}} - 1, \\
S &= S^{(B_{\mathrm{set}})}.
\end{aligned}$$

每层拥有独立的 CA、SA、FFN 和归一化参数。CA 的单头注意力矩阵形状为 $M \times NZ$，SA 的注意力矩阵形状为 $M \times M$。第一层查询由可学习参数 Q_M 初始化；后续层查询是上一层已经融合参考信息的风格 token。

拼接只是键值输入的整理，不含可学习参数，也不构成独立编码阶段。U **不进行自注意力，也不在集合汇总中被更新；每层 CA 都读取同一份** U。这里的“不更新 U”指前向计算中的信息流；训练梯度仍可通过每层键值分支回传至逐组编码器。补齐参考产生的描述在每一层 CA 中使用相同的键值掩码。Q_M 与 Q_Z 是不同的参数集合，即使 $M = Z$ 也不共享。

汇总时不添加参考组序号、拼接后的绝对序列位置或新的空间网格位置。组内空间与上下文信息可以存在于描述内容中，但查询不再显式访问原始分组关系。这沿用阶段 B 的假设：压缩描述应足以支持内容驱动的集合汇总。

S 表示对整个参考集合的风格总结；其行数与 N 无关，但数值内容依赖全部有效参考。M 是表示容量，不等于预先定义的中文风格属性数量。参考分支只依赖 $\mathcal{C}$，不依赖目标字 $K_\star$、时间 t 或当前状态 x_t。因此同一组参考的 S 可以在多个目标字和各 flow 步之间复用。训练时不截断其梯度，参考分支与生成主干联合优化；训练更新后缓存需重新计算。

4 参考顺序无关性与可变数量输入

本任务要求参考对整体重排不改变输出；不允许仅重排 H_i 或仅重排 K_i。逐组编码与压缩使用共享参数，因此组重排只会重排 U 的整组行，得到 $U' = PU$。由于键值投影与归一化均逐 token 进行，对应有 $K' = PK$、$V' = PV$。任意固定查询对键和值的同步重排不变。以单头为例，

$$\mathrm{softmax}\left(\frac{Q(PK)^\top}{\sqrt{d_h}}\right) PV = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right) V.$$

第一层 CA 因而对参考重排不变；其后的 M token 自注意力、逐 token 前馈网络和残差也不引入参考顺序。以此逐层归纳，后续 CA 的查询同样不随参考重排变化，所以最终 S 不变。固定初始噪声和积分设置后，生成结果也不变，允许浮点计算误差。本稿暂不使用 dropout；若以后引入随机操作，需区分确定性推理与随机训练中的性质。

批处理中不同样本可以有不同 N。补齐参考组产生的全部 Z 个描述必须在最终交叉注意力中作为无效键值屏蔽。M 个查询均有效，但不得读取这些补齐 token。应避免给一个查询构造全部键均不可见的 softmax。$N = 1$ 时，M 个查询直接读取唯一一组的 Z 个描述，不需要特殊的跨组退化处理。

5 生成主干与 flow 起点

5.1 目标 patch 与主干结构

采用带风格交叉注意力的 DiT 主干。在每个时刻，仅将当前单通道图像 x_t 分块并投影：

$$T^{(0)} = \mathrm{PatchEmbed}_{\mathrm{gen}}(x_t) + P_{xy} \in \mathbb{R}^{L\times d}.$$

每个目标块的输入为 p^2 维，投影为 d 维；主干始终保留 $L = (n/p)^2$ 个空间 token。目标标准字 $K_\star$ 仅参与构造起点 x_0，不再独立输入速度网络。参考分支仍将 H_i, K_i 成对拼接，参考分支与目标分支的分块投影不共享参数。每个 block 依次执行目标自注意力 SA、读取 S 的交叉注意力 CA，以及逐 token FFN。此处 SA、CA 表示独立的注意力子层，FFN 单独列出；不重复套用参考分支中包含 FFN 的复合块定义。

时间调节只施加于 SA、FFN，以及输出头。 SA 与 FFN 沿用原始 DiT 的 AdaLN-Zero 配置；新增 CA 采用普通前置 LN 与残差，不另外生成时间缩放、偏移或门控。不同 block 各自将同一个 S 投影成键和值。CA 的查询来自已经受时间影响的主干特征，因此仍然能够随 t 改变读取方式。

5.2 时间输入：标准 AdaLN-Zero 与普通 CA

连续时间 $t \in [0, 1]$ 经正弦余弦编码及两层线性映射（中间使用 SiLU）得到共享的 $e(t) \in \mathbb{R}^d$。编码的时间尺度在训练与推理时保持一致；t 表示流的位置，不是积分步号或步长。每个 block 使用自己的一次映射：

$$(a_s, b_s, g_s, a_f, b_f, g_f) = \mathrm{Split}_6\Big(W^{(b)}_{\mathrm{mod}}\,\mathrm{SiLU}(e(t)) + c^{(b)}_{\mathrm{mod}}\Big), \qquad W^{(b)}_{\mathrm{mod}} : \mathbb{R}^d \to \mathbb{R}^{6d}.$$

下标 s, f 分别表示 SA 与 FFN。六个向量均为 d 维，在该样本的全部 L 个 token 间广播；不同 block 使用不同参数，同一 block 内通过一个线性层一次性生成六组输出。定义

$$D_j(T, t) = (1 + a_j(t)) \odot \mathrm{LN}_j(T) + b_j(t), \qquad j \in \{s, f\}.$$

受时间调节的 LN 不再带自身的可学习仿射参数。一个 block 的完整更新为

$$\begin{aligned}
A &= D_s(T, t),\\
T_s &= T + g_s(t) \odot \mathrm{MHA}_s(A, A, A),\\
T_c &= T_s + \mathrm{MHA}_c(\mathrm{LN}_q(T_s), \mathrm{LN}_{kv}(S), \mathrm{LN}_{kv}(S)),\\
T^+ &= T_c + g_f(t) \odot \mathrm{FFN}(D_f(T_c, t)).
\end{aligned}$$

CA 中的 Q、K、V 投影仍然彼此独立；注意力矩阵为 $L \times M$。主干不直接读取 R_i 或 U_i，参考表示 S 本身不接受时间调节。

$W_{\text{mod}}^{(b)}$ 及其偏置零初始化，因此 SA、FFN 的缩放增量、偏移与残差门初始均为零；门控向量通过训练学习，不使用额外 sigmoid。新增 CA 保留普通残差，不引入额外零门，所以此扩展 block 整体不保证初始恒等；AdaLN-Zero 的零残差初始化作用于 SA、FFN 两个分支。

输出头另用 $\text{SiLU} + \text{Linear}(d, 2d)$ 生成 $(a_{\text{out}}, b_{\text{out}})$，对最终 $L \times d$ 特征做时间调节的归一化，再用所有 token 共用的线性层 $d \to p^2$：

$$L \times d \xrightarrow{\text{时间调节 LN + 共享线性层}} L \times p^2 \xrightarrow{\text{按原二维位置拼回}} \widehat{v}_t \in \mathbb{R}^{n \times n \times 1}.$$

沿用原始 DiT 的输出头初始化，时间调节映射与最终 patch 线性映射均零初始化。输出矩阵只有 $d \times p^2$ 个权重，不随 L 增长；不存在把整图压成一个 token 后再解码的步骤。输出是当前灰度状态的像素速度，而不是最终字图；速度网络记作 $v_\theta(x_t, t, S)$，输入与输出图像均为单通道。

5.3 训练路径：标准字加噪声到手写真值

本稿暂在像素空间定义 flow，不额外引入图像自编码器。主干内部的 patch 特征不是独立的生成状态；若将来改为 latent flow，需另行定义共同编码空间与解码器。

取固定实验超参数 $\sigma \geq 0$（默认 0.1），独立采样 $\epsilon \sim \mathcal{N}(0, I)$ 和 $t \sim \mathcal{U}(0, 1)$，定义

$$\begin{aligned} x_0 &= K_\star + \sigma\epsilon, \\ x_t &= (1-t)x_0 + tH_\star, \\ u &= H_\star - x_0, \\ \mathcal{L}_{\text{flow}} &= \mathbb{E}\left[\frac{1}{n^2}\left\|v_\theta(x_t, t, S) - u\right\|_F^2\right]. \end{aligned}$$

同一个 x_0 同时用于构造 x_t 和监督速度 u。目标标准字 $K_\star$ 通过起点及训练状态影响速度预测，不作为独立通道提供；$H_\star$ 仅通过训练状态与监督目标参与训练。参考分支的所有可学习查询、编码器和生成主干均由 flow 损失联合训练。训练器还默认加入作者风格损失，完整目标见下一小节；当前没有笔画标注或图像重建损失。

这里的加噪是以标准字为均值的加性高斯扰动，不是把图像与噪声做权重和为 1 的混合。x_0 可能超出 $[-1, 1]$；训练状态和积分中间状态均不裁剪。训练和推理必须使用相同的起点分布；不能仅在推理时替换纯噪声起点。当前像素线性插值定义的是训练路径，不保证中间状态是自然字形，也不把 t 定义为手写风格强度。

5.4 作者风格损失与训练、验证口径

`author_style_loss` 比较一个微批次内不同样本的固定风格槽位。设第 i 个样本的作者索引为 a_i，其第 m 个风格 token 为 $S_{i,m}$。先转为 float32，并沿特征维进行 L2 归一化：

$$\bar{S}_{i,m} = \frac{S_{i,m}}{\max(\|S_{i,m}\|_2, 10^{-8})}, \qquad c_{ijm} = \bar{S}_{i,m}^\top \bar{S}_{j,m}.$$

定义有序配对集合 $P_s = \{(i,j) : i \neq j, a_i = a_j\}$ 和 $P_d = \{(i,j) : a_i \neq a_j\}$，分别取平均：

$$\begin{aligned} \mathcal{L}_s &= \frac{1}{|P_s|M} \sum_{(i,j)\in P_s} \sum_m (1 - c_{ijm}), \\ \mathcal{L}_d &= \frac{1}{|P_d|M} \sum_{(i,j)\in P_d} \sum_m c_{ijm}^2, \\ \mathcal{L}_{\text{author}} &= \mathcal{L}_s + \mathcal{L}_d, \\ \mathcal{L}_{\text{train}} &= \mathcal{L}_{\text{flow}} + \lambda\mathcal{L}_{\text{author}}, \qquad \lambda = 0.05\ (\text{默认}). \end{aligned}$$

空配对集合对应项取保持梯度连接的零，不做除零；单样本批次作者项为零。函数还接受同作者、异作者项的独立非负权重，默认均为 1，训练器采用默认值。同作者项鼓励一致，异作者项鼓励同槽方向正交，不

能把相反方向当作零损失解。作者项不保证内容与风格严格解耦，参考采样也不保证每个 batch 都有同作者配对。配对仅在每个微批次内部进行，梯度累积不增加配对范围；设 `author_style_loss_weight=0` 可关闭这一项。

一个微批次只编码一次风格，同时用于速度预测和作者项。训练 MSE 以 float32 计算；作者 token 的归一化使用 float32，注意力和相似度运算仍受当前 autocast 上下文影响。EMA 模型在挂载训练专用作者权重之前复制，因此 `train_loss` 为上述总目标，而 `val_loss_ema` 只统计 EMA 的 flow MSE。最优检查点依据后者选择，不能把两个指标当作完全相同的损失口径。

5.5 推理与随机性

先计算 S，再采样 $x_0 = K_\star + \sigma\epsilon$，固定 S 并求解

$$\frac{dx_t}{dt} = v_\theta(x_t, t, S), \qquad t : 0 \longrightarrow 1.$$

暂采用 J 步等步长 Euler 积分：

$$t_j = j/J, \qquad x_{t_{j+1}} = x_{t_j} + \frac{1}{J} v_\theta(x_{t_j}, t_j, S).$$

`sample_euler` 在 `no_grad` 下仅在最后把 x_1 裁剪至 $[-1, 1]$，返回归一化图像张量；灰度映射由预览函数或调用方完成，得到 $\widehat{H}_\star$。除初始噪声外，生成过程不引入额外随机量。各步只能从当前状态读取目标字信息，不再额外读取干净目标标准字。生成输入层由双通道改为单通道；旧版双通道检查点不能直接续训，当前训练检查点版本为 2。

当参考、目标和噪声固定时，输出确定。$\sigma = 0$ 可另作确定性迁移对照，但同一条件下的确定性 ODE 无法产生多个不同写法。更大 σ 不自动意味着更多合理变化；若需要推理时可调的噪声强度，应另行设计覆盖不同强度的训练和相应条件输入。采样函数允许显式传入 `noise` 和覆盖 `sigma`，但覆盖只改变起点，模型没有噪声强度条件输入，也没有保证不同强度下的生成效果。

6 计算成本与压缩的实际含义

以下仅统计注意力中计算权重与加权求和的主要项，并将重复层数显式写出；忽略常数。投影和前馈网络的成本通常还包含“token 数量乘 d^2”的项。

操作	主要注意力成本
N 组分别进行 L 个 patch 的自注意力	$O(NL^2d)$
每组用 Z 个查询读取 L 个特征	$O(NZLd)$
B_{set} 个集合汇总层：CA 读取 NZ 个描述并对 M 个 token 做 SA	$O\big(B_{\text{set}}(MNZ + M^2)d\big)$
一个主干层的目标自注意力	$O(L^2d)$
一个主干层读取风格条件	$O(LMd)$

计入参考分支的实际层数，本稿默认结构的主要注意力成本为

$$O\big(B_{\text{local}}NL^2d + NZLd + B_{\text{set}}(MNZ + M^2)d\big)\,.$$

参考描述不做全局自注意力，故不包含 $O(N^2Z^2d)$ 或 $O((Nq^2)^2d)$ 的计算项；新增自注意力只发生在固定数量的 M 个风格 token 上。固定 $L, Z, M, d, B_{\text{local}}, B_{\text{set}}$ 时，参考分支的上述计算成本仍随 N 线性增长。若不做逐组压缩，而让每层的 M 个查询直接读取全部 NL 个组内特征，集合 CA 成本为 $O(B_{\text{set}}MNLd)$。压缩后单层及全部层的 CA 读取项之比均为 Z/L；例如 $L = 256, Z = 16$ 时，该项变为 $1/16$。这一比较不包含新增的逐组压缩成本，不能据此宣称整个参考分支加速 16 倍。

参考编码不在 flow 积分循环内重复执行；相同参考可复用其输出。若主干有 B_{gen} 层、采样有 J 步，其注意力成本约为 $O(JB_{\text{gen}}(L^2 + LM)d)$。实际训练显存和时间还取决于批量大小、反向传播、前馈宽度与注意力实现。FlashAttention 可降低中间矩阵的显存开销；代码调用 PyTorch 的多头注意力，具体后端取决于设备、版本和精度，不保证始终使用 FlashAttention。本版关于 N 的线性计算规模来自结构本身。实

际 batch 将参考补齐至最大槽数，补齐组仍参加逐组编码，计算成本应按槽数而非仅有效参考数估算。采样只缓存风格 token；主干每步仍计算各层的风格 LN 与 K/V 投影，没有 K/V cache。

压缩的代价同样明确：某种少见笔画或部件证据若在生成 U_i 时已经丢失，后续 M 个查询只能利用保留下来的信息，不能通过重新读取 R_i 找回它。因此 Z 控制单字证据容量，M 控制集合汇总宽度，B_{set} 控制反复读取和风格 token 交互的深度，三者应分别考察。

7 默认配置与数值实现

7.1 模型参数

当前模型使用通道在前的 PyTorch 张量。参考为 $[B, N, C, n, n]$，流状态和速度为 $[B, C, n, n]$，时间为 $[B]$，风格为 $[B, M, d]$。数据与训练器只支持灰度 $C = 1$；底层模型可按配置构造其他通道数。

`ModelConfig` 字段	默认值	含义
`image_size / patch_size`	64/8	$q = 8, L = 64$
`in_channels`	1	参考拼接为两通道，主干为单通道
`hidden_size / num_heads`	256/8	每头维度 32
`mlp_ratio`	4.0	FFN 中间维度 1024
`local_depth`	4	逐组 SA–FFN 层数
`set_pool_depth`	2	集合 CA–SA–FFN 层数
`generator_depth`	8	主干层数
`description_tokens`	32	每组描述数 Z
`style_tokens`	32	风格槽数 M
`time_frequency_size`	256	时间 Fourier 特征维数
`time_scale`	1000	编码前将连续时间放大
`sigma`	0.1	加性高斯噪声强度，允许 0

分块使用带 bias 的 `Conv2d`，kernel 和 stride 均为 p。图像边长需可被 p 整除，d 可被头数和 4 整除；时间特征维需为正偶数。集合层数至少为 1，逐组及主干层数允许为 0。默认模型全部可学习参数为 17,984,320，其中参考编码器 6,106,624，生成主干 11,877,696。`count_parameters.py` 可用模型字段的 kebab-case 选项单独统计其他结构，不改变训练配置。

二维位置按行、列各占 $d/2$ 维，每轴拼接 sin、cos；时间编码拼接 cos、sin。频率均以 10000 为周期基数。时间编码第 k 个频率为

$$f_k = \exp\left(-\log(10000)\frac{k}{F/2}\right), \qquad e_{\text{raw}}(t) = [\cos(t\,\tau\,f_k); \sin(t\,\tau\,f_k)],$$

其中 F 为 `time_frequency_size`，τ 为 `time_scale`。

位置与时间频率为非持久化 buffer，不写入 `state_dict`。时间特征维的默认值在类定义时确定为 256，单独修改 d 不会自动修改 F。

全部注意力为 batch-first 的 `nn.MultiheadAttention`，dropout 为 0，Q/K/V 独立投影并带 bias，调用时不返回注意力权重。普通 LN 带可学习仿射，$\varepsilon = 10^{-5}$；AdaLN 用无仿射 LN，$\varepsilon = 10^{-6}$。FFN 内宽为 $\lfloor d \cdot \texttt{mlp_ratio} \rfloor$，激活为 GELU。$Q_Z, Q_M$ 独立以标准差 0.02 的正态分布初始化；参考输出没有额外末尾 LN。时间调制层和最终速度投影零初始化，其余层采用 PyTorch 默认初始化。最终投影为零意味着初始速度为零，不能据此将包含普通 CA 的整个主干块视为恒等映射。

7.2 训练参数

TrainingConfig 字段	默认值	含义
steps	100000	优化器总步数
batch_size / accumulation_steps	64/1	微批量 / 累积次数
min_references / max_references	1/16	参考数均匀整数采样范围
learning_rate	10^{-4}	基础学习率
min_lr_ratio / warmup_steps	0.1/1000	最低比例 / 预热步数
weight_decay / gradient_clip	0.01/1.0	权重衰减 / 全局 L2 阈值
ema_decay	0.999	EMA 衰减
author_style_loss_weight	0.05	作者项权重
precision	bf16	CUDA 默认精度
preload_to_gpu / workers	False / 8	字图常驻显存 / 训练 worker 数
val_batches / validate_every	16/1000	验证批次 / 间隔
save_every / log_every	500/50	保存 / 日志间隔
sample_steps / seed	32/42	预览 Euler 步数 / 种子

新训练直接读取 src/config.py，不读取额外配置文件；有效批量为微批量乘累积次数，每个微批损失除以累积次数后反向。训练使用 AdamW，betas 为 $(0.9, 0.95)$，全部参数在同一组内进行权重衰减。学习率按从 1 开始的优化器步数线性预热，再余弦衰减至基础值的最低比例。--stop-after 只提前停在绝对步数，不改变总日程。CUDA 支持 BF16/FP16/FP32 并启用 TF32；CPU 实际使用 FP32，只有 FP16 启用 GradScaler。

8 数据准备、划分与确定性采样

从 DiT-flow/ 内运行 python -m src.prepare_data，必填参数为 --gnt-root、--font、--output。可选默认值为字体索引 0、图像边长 64、边距 4、最少字符数 17、验证比例 0.1、种子 42；其中边长读取模型默认值，最少字符数为默认最大参考数加一。输出目录必须新建或为空，代码不自动区分官方训练/Test 文件。

递归排序读取 GNT，每个文件视为一个作者，文件 stem 为 ID，要求至少两个文件且 ID 不重复。GNT 数值字段按小端解析，两个原始标签字节按 GBK 解码；截断、非法记录尺寸或标签直接报错。保留基本 CJK 范围 U+4E00–U+9FFF 内的单字，并用字体 cmap 排除缺字；每位作者每个字符仅保留首次成功处理的图像，空白及字符数不足作者计入跳过统计。手写和标准图分别紧裁白背景、保持宽高比，用 LANCZOS 缩放并在白色方形中居中。标准图先以四倍目标边长的字号渲染；默认预处理没有随机图像增强。

按作者用独立种子打乱后划分 train/val。作者数为 W 时，验证作者数为

$$\min\big(W-1, \max(1, \mathrm{round}(W \cdot \texttt{val_fraction}))\big).$$

作者不跨集合，但同一字符类别可以在两侧出现。输出为 uint8 的作者 NPY、标准字 NPY，以及最后写入的 manifest.json 完成标志。数据格式版本为 1，与训练检查点版本 2 是不同概念。manifest 记录字符索引、作者文件/标签/划分、尺寸、边距、字体和像素文件哈希、种子及跳过统计。

训练启动逐作者加载全部 train/val 像素，只保留一份 uint8 字图。CPU 使用 RawArray，fork/spawn worker 和训练/验证共享存储；采样不再读取 NPY。像素占用为 (手写数 + 标准数)n^2 字节，不含模型与浮点批次。显存预加载只允许 CUDA，并强制训练 worker 为 0。

每个 episode 用 random.Random(f"{seed}:{index}") 独立采样：均匀选作者、均匀选参考数，再无放回选择 $N+1$ 个不同字符，首个作为目标。像素转为 pixel/127.5 − 1，即白 +1、黑 −1；参考补至最大槽数，补齐值为白色，并提供布尔有效掩码和稳定的 manifest 作者索引。至少需要最大参考数加一个不同字符；按作者均匀抽样不等于按全部字图均匀抽样。掩码省略时全部槽位有效，每个样本至少一个有效槽位；应使用有限数值填充补齐图像。

episode 仅由索引和种子决定，不受 worker 数和预取进度影响。续训从已完成步数乘有效批量继续全局索引；验证使用固定索引和训练种子加一。数据指纹为 manifest 原始字节的 SHA-256，空白变化也会导致续训拒绝。加载器校验 NPY 的 dtype/shape 和标签索引，但不重新核验记录的 NPY/字体内容哈希。

9 验证、诊断与检查点恢复

9.1 EMA、验证和退出

每次成功优化器更新后，EMA 参数按 $\theta_{\text{ema}} \leftarrow \rho\theta_{\text{ema}} + (1-\rho)\theta$ 更新。每隔验证间隔及计划结束步使用 EMA 做验证与预览；正常提前结束也验证。验证用隔离的 PyTorch RNG 和训练种子加 10000 固定时间/噪声，按样本数平均 flow MSE。预览在全部验证损失之后采样，展示第一批最多四个样本的所有参考槽、标准字、真值和生成图。验证 worker 固定为 0；更改设备或精度不保证跨设备逐位一致。

训练循环接收 SIGINT/SIGTERM 后完成当前优化器步、跳过该步验证，并保存最近状态。强杀或一般运行异常不会额外保存故障当前步，只保留已成功写入的检查点。

9.2 梯度与日志

AMP 反缩放后记录裁剪前全局 L2 范数，再安全裁剪；float32 范数归约出现非有限值时用 float64 重算，区分归约溢出与真实异常梯度。真实异常梯度不更新参数：FP16 降低 scale 并复用相同数据、时间与噪声，最多尝试 16 次；BF16/FP32 直接报错停止，非有限损失也立即停止。诊断在启动/续训首步、日志间隔、计划结束步及异常/float64 回退时记录，包括最大的八个参数及直接所属模块、最大元素、裁剪比例、AMP scale、尝试序号和异常参数名。模块排行不是整个参考分支或主干的递归总范数；Inf/NaN 用字符串保存为合法 JSON。

`metrics.jsonl` 追加训练总损失、学习率、原始梯度范数、裁剪乘数及可选 EMA 验证损失。普通数字保留八位小数，极大/极小数使用科学计数法；耗时为带秒单位字符串，CUDA 峰值显存为带 `GB` 字符串，但分母是 1024^3，实际单位为 GiB。显存字段来自 `max_memory_allocated`，不是 reserved memory 或单步独立峰值。`run.json` 记录设置、数据指纹、设备、实际精度、torch 版本与运行预加载方式，续训时重写。

9.3 保存与续训

从 `DiT-flow/` 内运行 `python -m src.train`。必填参数为 `--data`、`--output`。

可选参数为 `--resume`、`--device`、`--workers`、`--stop-after`。设备默认 `auto`，选择 CUDA 或 CPU，支持显式 CUDA 索引。仅支持 CPU 或单张 CUDA，没有分布式入口。训练文件也支持从仓库根目录按文件路径启动，参数路径相对工作目录。

检查点版本 2 保存模型和 EMA 两套权重、AdamW、GradScaler、CPU/选定 CUDA 的 RNG、模型/训练配置、数据指纹、已完成步数和最优验证值。`best.pt` 在 EMA flow 验证严格改善时写入，也含完整训练状态；`last.pt` 按保存间隔及正常结束/响应退出请求写入，可能落后于当前内存步数。恢复旧版双通道版本 1 会在输出写入前报错；已准备的灰度数据可以复用。

续训采用保存的模型和训练超参数，仅运行开关 `preload_to_gpu` 始终读取当前配置，不写入检查点 settings；设备、worker 数、提前结束步也可通过命令行改动。保存步数必须小于本次终点，不能用提前结束选项延长保存的总日程。新运行拒绝已有结果，只允许配置/指纹一致、仅有启动元数据和空预览目录的启动重试。续训可使用新目录；原非空目录须有一致元数据，且 `last.pt` 不能比恢复点更新。从更早的 `best.pt` 回退，应选择新的输出目录。

保存先完整写入 `.pt.tmp`，flush/fsync 后替换对应旧文件。I/O 错误最多尝试三次，间隔 1、2 秒，使用同一步内存状态，不重复训练；失败临时文件尝试清理，持续失败停止，普通序列化错误直接抛出。先保存刷新的 best，再按条件保存 last，两者不是整体事务。保存需要至少额外一份完整检查点的空间，磁盘/配额故障不能由重试保证修复。

10 验证范围

从 `DiT-flow/` 内运行 `python -m unittest discover -s tests -v`。现有测试覆盖结构/掩码/作者损失/flow、GNT 与真实字体预处理、共享内存无文件采样、参考数与目标排除、设备选择、日志、梯度异常、检查点写入失败，以及小模型 CPU 续训与连续训练一致性。CUDA 设备选择测试使用模拟；测试不能替代真实 GPU 性能、稳定性和生成质量实验。